

Astra and Grok: a simpler subagent system

Decision, implementation sequence, and acceptance gates

Prepared for Paul | 14 September 2026

Keep Astra in native Codex and reduce the local orchestrator to a small bridge for headless Grok workers. Astra owns task selection, instructions, review, and next steps. The bridge owns execution, durable results, history, and cancellation. Keep DeepSeek Harness as an alternative if a measured limitation in Codex prevents the required workflow.

Status: proposed implementation. Existing transport evidence supports feasibility; the new bridge still requires the validation below.

1. Scope and evidence

Required outcome. Astra can start a bounded Grok task, continue useful work, collect a meaningful result, inspect supporting history, and resume the same worker conversation for related follow-up. Concurrent writers use separate worktrees. Each PR remains atomic and passes CI.

Keep in the bridge	Move out of its mandatory path
Headless process launch; exact session IDs; result parsing; durable logs; cancellation; bounded concurrency.	Team registration, task acceptance ledgers, plan revisions, message acknowledgments, and publication routing.
Worktree identity supplied at launch; one active run per session; small status view.	Interactive tmux worker ownership; pause/resume choreography; embedded Graphite and CI workflow engines.

Verified locally. The current adapter supports Grok’s single-turn streaming JSON output and exact-session resume. The runner checks for a terminal result, successful process exit, and observed child-process release. The default Grok path nevertheless uses an interactive tmux worker and infers turn completion from session updates. These are source observations, not new execution tests. [L1]

Verified in recorded tests. Existing evidence reports successful Grok headless execution and same-session recall. It also reports that a queued message woke an idle interactive Codex CLI, while an exited headless coordinator did not self-wake. Treat those as version-specific observations. [L2]

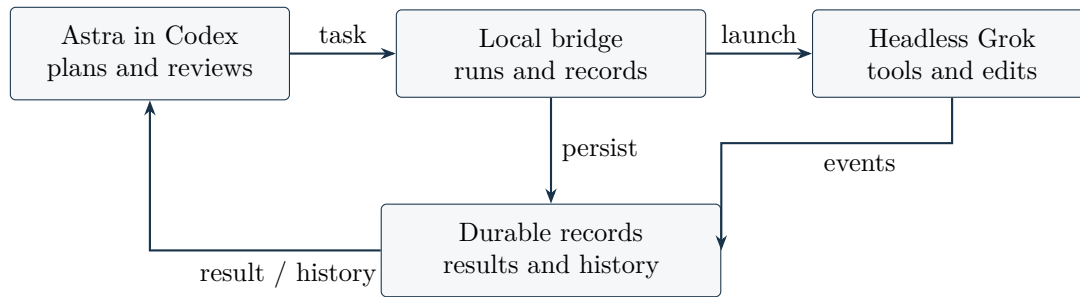
External design support. Codex documents delegation with separate contexts, follow-up instructions, and returned summaries. DeepSeek separates subagent execution from its optional team/task layer. Anthropic separates durable session history from replaceable execution. These support the proposed boundaries; none establishes that one harness is universally best. [S1–S3]

2. Proposed operating defaults

Start with **Astra high effort, Grok medium effort, at most three active workers, and one delegation level.** Use high effort for difficult debugging, complex implementation, and independent review. Reserve the highest supported setting for exceptional problems. These are pilot defaults, not benchmarked optima; effort labels are provider-specific. [S1, S4]

Give each worker an objective, relevant context, write scope, completion criteria, and expected report. Reuse its session for related fixes; start fresh for unrelated work. Astra may inspect code or perform a small dependent change itself. Add an independent reviewer when the risk warrants it.

3. Small interface and explicit ownership



Operation	Contract
<code>spawn</code>	Takes request ID, task, cwd, write scope, and effort. Returns session ID and run ID after launch intent is durable.
<code>wait</code>	Takes run IDs and an event cursor. Waits for at most 30 seconds; returns new terminal results and a cursor, or a pending response. Results remain inspectable.
<code>inspect</code>	Lists known sessions/runs or reads one run's state, final report, and a bounded history slice.
<code>resume</code>	Takes session ID and follow-up instructions. Creates a new run in that conversation; rejects a busy session.
<code>cancel</code>	Requests termination of the owned execution. Reports requested versus observed outcome until the run has settled.

Minimal records and result contract

A **session** holds its stable bridge ID, Grok session UUID, cwd/worktree identity, and ordered runs. A **run** holds its request ID, immutable task input, process identity, execution state, timestamps, terminal envelope, and references to result and history files. Keep raw provider output separate from the compact model-facing report.

The worker report contains **outcome** (completed, blocked, or partial), a summary, changes/findings, validation, unresolved issues, and artifact references. This is the worker's assessment. The runtime independently records how execution ended. Validate the report shape; malformed output remains available for inspection and is flagged as an invalid report rather than silently treated as success.

Lifecycle and retry rules

Runs move from **starting** to **running**, then **completed**, **failed**, or **cancelled**. **Interrupted** records an execution whose terminal outcome cannot be established after recovery. Completed execution requires a valid terminal provider envelope, normal exit, and release of observed child processes; Astra still evaluates correctness.

Persist input before launch and the result before exposing completion. Repeating a request ID with identical input returns the original run; changed input is rejected. An ambiguous crash never triggers blind re-execution. Inspect the saved history and worktree before a new attempt. Cancellation does not undo edits or remote effects. Resume changes conversation state, not the checked-out code.

4. Implementation as four atomic PRs

Build beside the current runtime, initially disabled, using the existing Python toolchain. Reuse proven components and one authoritative metadata store. Keep artifacts outside the repository. Use `gt create` and `gt submit --no-interactive`; verify CI at each PR head before acceptance.

PR 1: Extract the headless execution adapter

Extract launch and terminal-result parsing from `adapters.py` and the headless portion of `runner.py`. Use prompt files, exact session UUIDs, and the current authorized execution policy. Add a standalone path and deterministic fake provider; preserve legacy behavior.

Acceptance: a bounded task produces exactly one final report; commentary and partial messages cannot masquerade as completion. Verify normal completion, provider error, missing terminal envelope, malformed report, and nonzero exit. All existing affected tests and CI pass.

PR 2: Add durable run ownership and continuation

Add persistent session/run records, bounded history reads, request deduplication, one active run per session, and explicit resume. Reuse relevant process identity and child cleanup code. A small supervisor owns workers independently of a transient tool connection. Reject conflicting writers and excess concurrency clearly; avoid adding a task scheduler.

Acceptance: reconnecting clients can collect saved results; a crash never creates an automatic duplicate run; same-session continuation retains context; cancellation waits for observed cleanup or reports uncertainty. Tests exercise interruption before/after launch and before/after result persistence.

PR 3: Expose the bridge to Astra

Expose the five operations through local MCP with short coordinator/worker instructions. Return summaries and references; bound waiting calls. Astra keeps waiting or working while results are pending. MCP supports tool integration; automatic coordinator wake-up remains unproven. [S5]

Acceptance: in the actual Codex client Paul uses, Astra launches two Grok workers, remains steerable, collects both results, inspects a selected history slice, and resumes one worker without user relay. Verify client disconnect/reconnect. Do not claim native subagent UI integration or idle wake without observing it.

PR 4: Pilot, switch defaults, and retire legacy paths

Run the representative tasks on page 4. If acceptance gates pass, make the bridge the recommended workflow and simplify the local-orchestrator skill. Stop routing new work through the legacy acceptance, publication, and tmux machinery. Keep legacy records readable and finish or explicitly stop active legacy work before removal.

Acceptance: migration changes no unrelated configuration, loses no existing history, and has a documented rollback to the legacy workflow. Retain any small status view as an optional reader. Delete obsolete code only after active use and references are checked.

5. Validation and rollout decision

Exercise	Required evidence
Parallel read tasks	Both results reach Astra without manual relay; each history belongs to the correct session.
Scoped code change	Changes remain in the assigned worktree; relevant validation and the reviewed commit are recorded.
Related follow-up	Worker recalls prior task context using the exact saved Grok session; follow-up is a distinct run.
Blocked task	Worker returns a specific blocker or partial report; runtime distinguishes this from transport failure.
Failure / cancellation	No false success or automatic duplicate launch; owned process cleanup is observed or uncertainty is explicit.
Coordinator reconnect	Finished results survive disconnect; Astra can list and collect them after reconnecting.
Integrated change	Combined behavior passes appropriate tests; each PR is atomic and passes required CI.

Measure the whole workflow. Compare Codex alone with Codex plus Grok on matching starting revisions and instructions. Record time to verified completion, usage, retries, defects, conflicts, and human interventions. Compare medium/high worker effort. Treat the pilot as directional evidence.

Switch only when lifecycle tests pass, results reach Astra without manual copying, and the pilot improves the workflow without an observed correctness regression. If inactive-coordinator wake-up is required, verify it separately; a queue receipt is insufficient.

Reconsider DeepSeek when required unattended continuation or custom UI cannot be achieved through Codex. Evaluate an upstream Grok plugin first; fork only for a demonstrated missing extension point. DeepSeek’s current Codex provider lacks session persistence/resume. [S2, S6]

Evidence and source notes

Inspected 14 September 2026. Upstream behavior can change. Local baseline: `dab9548014d6`, `nthpaul/dotfiles`; unrelated working-tree changes excluded.

L1. Local implementation. Root: `codex/.codex/orchestrator/`. Files: `adapters.py`, `runner.py`, and `schema.sql` in its `orchestrator/` directory; root `README.md`.

L2. Local transport evidence. `docs/transport-evidence.md`, sections “Real runs,” “Parser contract,” and “Unsupported or uncertain behavior.” Records Codex CLI 0.154.0 and Grok 1.0.30 observations.

S1. OpenAI, Subagents: context isolation, model/effort configuration, and orchestration controls.

S2. DeepSeek, Subagent subsystem and Architecture: result semantics, provider interfaces, and optional Agent Teams.

S3. Anthropic, Scaling Managed Agents: Decoupling the brain from the hands, 8 April 2026: independent session history and execution.

S4. DeepSeek, Subagent delegation tool: provider-owned effort settings and foreground/background scheduling.

S5. OpenAI, Model Context Protocol: local Codex tool integration.

S6. DeepSeek, Codex provider, “Known Limitations and Deferred Work”: fresh ephemeral runs without continuation or product-session persistence.